will become entranced by a noisy TV, forever staring at it. This is because it is intrinsically rewarded by unpredictability, and white noise is very unpredictable.

Figure 8.6. Diagram of the forward-prediction module function, f: (St,at) → Št+1, which maps a current state and action to a predicted next state.

Figure 8.7. The inverse model takes two consecutive states and tries to predict which action was taken.

Figure 8.8. The encoder model takes a high-dimensional state representation such as an RGB array and encodes it as a low-dimensional vector.

Figure 8.9. The forward-prediction module actually uses encoded states, not the raw states. Encoded states are denoted φ(St) or .

Figure 8.10. The curiosity module. First the encoder will encode states St and St+1 into low-dimensional vectors, φ(St) and φ(St+1) respectively. These encoded states are passed to the forward and inverse models. Notice that the inverse model backpropagates to the encoded model, thereby training it through its own error. The forward model is trained by backpropagating from its own error function, but it does not backpropagate through to the encoder like the inverse model does. This ensures that the encoder learns to produce state representations that are only useful for predicting which action was taken. The black circle indicates a copy operation that copies the output from the encoder and passes the copies to the forward and inverse models.

Figure 8.11. Each state given to the agent is a concatenation of the three most recent (grayscale) frames in the game. This is necessary so that the model can have access to not just the position of objects, but also their direction of movement.

Figure 8.12. The DQN architecture we will use. The state tensor is the input, and it is passed through four convolutional layers and then two linear layers. The ELU activation function is applied after the first five layers but not the output layer because the output needs to be able to produce arbitrarily scaled Q values.